

INFORME DE ACTIVIDADES DIARIAS

Proyecto: VitalIA Hub

Colaborador	Cristian David López Valencia
Fecha	17 de agosto de 2026
Proyecto	VitalIA Hub — Plataforma de IA médica
Horas registradas	8 horas

1. Resumen ejecutivo

Durante la jornada de hoy se dio el salto del prototipo local de VitalIA Hub a una arquitectura híbrida de nivel de producción. Se integró la interfaz ya construida con un backend profesional desplegado en la nube, manteniendo el hardware local como motor de inteligencia artificial. Al cierre del día los tres servicios (base de datos, backend y frontend) están activos y estables en Railway, junto con el canal seguro que conecta la nube con el modelo de IA local.

2. Arquitectura implementada

Se construyó una arquitectura híbrida cloud + IA local, combinando servicios gestionados en Railway con el motor de inteligencia artificial ejecutado en hardware propio:

- Frontend (React + Vite): se rescató la interfaz original del hackathon (chat médico y panel de triaje) y se actualizó App.jsx para apuntar dinámicamente al backend en la nube; se ajustó el indicador de estado del motor Ollama para consumir el endpoint /health. Desplegado como servicio estático en Railway.
- Backend (FastAPI, Python 3.12 — med-ai-hub-v2): entorno nuevo enfocado en rendimiento y asincronía. Incluye endpoints mock de autenticación (/api/auth/login, /api/auth/me) para compatibilidad retroactiva con el frontend del hackathon, y un evento de arranque (startup_event) que usa SQLAlchemy para crear automáticamente las tablas necesarias. Dockerizado y alojado de forma permanente en Railway.
- Base de datos (PostgreSQL gestionado): se reemplazó SQLite (local y efímero) por una instancia PostgreSQL en Railway, con la variable DATABASE_URL inyectada de forma segura entre contenedores y driver asíncrono asyncpg para evitar cuellos de botella con múltiples consultas concurrentes.
- Capa de IA local (Ollama + Cloudflare Tunnels): el servidor web vive en la nube, pero el modelo de IA vive en el hardware local. Un túnel cifrado de Cloudflare (.trycloudflare.com) conecta Railway con la instancia local de Ollama; el modelo Llama 3.1 (8B) procesa cada consulta en la GPU local y devuelve la respuesta a la web en tiempo real mediante streaming (Server-Sent Events).

Esta arquitectura híbrida se eligió porque mantiene el costo de servidores en \$0 sin sacrificar la privacidad de los datos: al procesar los modelos de IA en hardware local, los datos médicos nunca se envían a proveedores externos (OpenAI, Google, etc.), cumpliendo estándares de privacidad médica, mientras que la aplicación web permanece disponible 24/7 para cualquier paciente desde internet.

3. Desglose de actividades (8 horas)

Actividad	Duración
Recuperación y adaptación del frontend original del hackathon (med-ai-hub, React + Vite): actualización de App.jsx para que las peticiones HTTP apunten dinámicamente al nuevo backend en la nube en lugar de al entorno local, y ajuste del indicador de estado del motor Ollama para consumir el endpoint /health.	1h 30m
Construcción del nuevo backend (med-ai-hub-v2) en FastAPI sobre Python 3.12: creación de endpoints mock de autenticación	1h 30m

Actividad	Duración
(/api/auth/login, /api/auth/me) para compatibilidad retroactiva con el frontend del hackathon, y configuración de un evento de arranque (startup_event) con SQLAlchemy para auto-migración de tablas.	
Provisión y conexión de base de datos PostgreSQL gestionada en Railway: inyección segura de la variable DATABASE_URL entre contenedores e implementación del driver asíncrono asyncpg para soportar consultas concurrentes sin cuello de botella.	1h 00m
Diseño e implementación de la arquitectura híbrida cloud + IA local: configuración de un túnel cifrado con Cloudflare Tunnels (trycloudflare.com) para conectar el backend en Railway con la instancia local de Ollama, resolviendo los fallos iniciales de conexión del túnel.	1h 30m
Implementación del flujo de inferencia en tiempo real: streaming de respuestas del modelo Llama 3.1 (8B) vía Server-Sent Events desde Ollama local hasta el frontend, y corrección de fallos de healthcheck del backend en Railway (endpoint, timeouts y orden de arranque de servicios).	1h 00m
Dockerización y despliegue final del backend en Railway, pruebas end-to-end del flujo completo (frontend → backend en la nube → túnel Cloudflare → Ollama local → respuesta al paciente) y validación de estabilidad de los tres servicios (Postgres, backend, frontend) en estado Online.	1h 30m

4. Incidencias resueltas

Durante la construcción de la arquitectura híbrida se presentaron tres tipos de fallos, todos resueltos en el transcurso de la jornada:

- Túnel de conexión (Cloudflare): fallos en la comunicación entre el backend en Railway y la instancia local de Ollama a través del túnel, corregidos ajustando la configuración de red y las variables de entorno hasta lograr un canal estable y cifrado.
- Healthcheck de Railway: el servicio backend (med-ai-hub-v2) no superaba la verificación de salud al desplegarse; se ajustó el endpoint /health y los tiempos de espera hasta lograr un estado estable en Railway.
- Conexión a base de datos: errores en la inyección de la variable DATABASE_URL y en la migración inicial de tablas vía SQLAlchemy, resueltos y validados con Postgres, backend y frontend en estado 'Online'.

5. Estado final al cierre de la jornada

Al finalizar el día, el sistema es estable y operativo de extremo a extremo: un paciente puede escribir en la web, el mensaje viaja de Railway a través del túnel de Cloudflare hasta la instancia local de Ollama, que procesa el modelo Llama 3.1 (8B) y devuelve la respuesta en tiempo real por streaming. Servicios activos en Railway:

- Postgres — Online (con volumen persistente postgres-volume)
- frontend (vitalia.up.railway.app) — Online
- med-ai-hub-v2 (backend) — Online

6. Siguiendo pasos identificados

Como parte del cierre de la jornada se dejaron identificados los siguientes puntos pendientes para asegurar la estabilidad permanente del sistema:

- Adquirir un dominio propio y configurar un túnel persistente de Cloudflare, dado que el túnel actual (.trycloudflare.com) es temporal.

7. Conclusión

El avance del día corresponde íntegramente a las 8 horas de labor: migración del prototipo local a una arquitectura híbrida de producción, integrando frontend, backend, base de datos gestionada y el canal seguro hacia el motor de IA local, junto con la resolución de los tres bloqueos técnicos (túnel, healthcheck y base de datos) que impedían que el sistema quedara operativo.

Cristian David López Valencia